

Breadth-First Search

Queues, shortest routes, and DFS vs. BFS

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 12 Thursday, October 8, 2026

Goals for Today

- Explain why recursive DFS can return a route that is **not the shortest**.
- Describe BFS: explore a maze in **layers**, using a **queue**.
- Trace BFS by hand: the queue, the visited marks, and the **parent table**.
- **Design the parent table**: what it stores, its shape, and how to read and write one entry.
- **Rebuild the route** by following parent links from the end back to the start.
- Compare DFS and BFS: route, container, time and space.
- See how DFS_Solver and BFS_Solver are used through a Maze_Solver*.

Where We Are

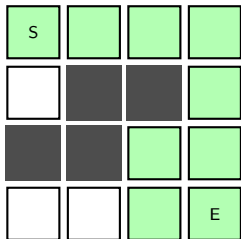
Date	What
Today, Thu 10/8	Class 12: BFS. Exercise 5 is due.
Tue 10/13	No class (Tuesday follows the Monday schedule)
Thu 10/15	Class 13: Midterm Review
Project 2	Task A (recursive DFS) was Class 11. Task B (BFS) is today.

Recap from Class 11

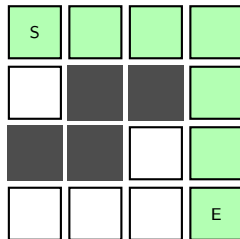
Recursive DFS asks “is there a path from here?”, marks each cell visited, and tries left, right, up, down. The **call stack** remembers the current route. Backtracking is a **return**.

DFS Found a Route. Is It the Shortest?

The 4×4 maze from the Project 2 handout. Black cells are walls; $S = (0, 0)$, $E = (3, 3)$.



DFS: 9 cells



BFS: 7 cells

DFS commits to a branch and returns the **first** route it finds. When we need the route with the **fewest cells**, we need a different search order.

Your Turn: How Many Steps from S ?

	0	1	2	3
0	S			
1				
2				
3				E

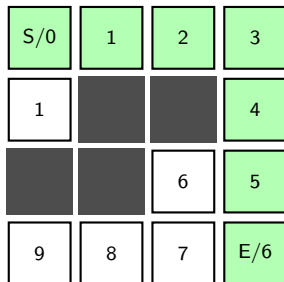
Gray: wall. Small numbers: row and column.

Imagine BFS spreading out from S like ripples in a pond. Each move goes one cell left, right, up, or down.

On your handout, write in each open cell the number of steps from S to that cell. That number is the cell's layer.

- S is layer 0, so write 0 there.
- Which cells are in layer 1? In layer 2?
- What is the layer of E ?

The BFS Idea: Explore in Layers



Numbers: steps from S . Green: the route.

Picture a stone dropped in a pond. The ripples reach every cell 1 step away, then every cell 2 steps away, then 3, and so on.

- Layer 0: S .
- Layer 1: $(0, 1)$ and $(1, 0)$.
- Layer 2: $(0, 2)$. Layer 3: $(0, 3)$
- The **first time** a ripple reaches E , it got there in the fewest steps.

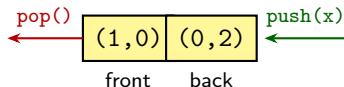
Question: what data structure hands cells back in the order they were discovered?

The Answer: a Queue

- A **queue** is first in, first out (Class 10): cells discovered earlier are explored earlier, so distance d is finished before distance $d + 1$.
- DFS uses a **stack** (the call stack): the *newest* cell is explored first, so the search runs deep.
- BFS uses a **queue**: the *oldest* waiting cell is explored first, so the search spreads out.

	0	1	2	3
0	S			
1				
2				
3				E

Blue: explored. Yellow: in the queue.



The queue after BFS dequeued S and then $(0, 1)$. In C++: `std::queue<Coord>`, with `push()`, `front()`, `pop()`, `empty()`.

BFS in Pseudocode

- 1 If the start or the end is not open, return an empty stack. Otherwise, if $\text{start} = \text{end}$, return a one-cell stack.
- 2 Make a queue q . Mark the start visited and push it.
- 3 While q is not empty, take the front cell cur and pop it.
- 4 For each neighbor nb of cur (any fixed order): if nb is open and not visited, mark nb visited **now** (when enqueued), set $\text{parent}[\text{nb}] = \text{cur}$, and push nb . **If nb is the end, stop the whole search** (leave both loops).
- 5 If the end was never reached, return an empty stack.
- 6 Otherwise walk from the end back to the start by parents, pushing each cell on a stack (the start ends on top).

When Does BFS Stop?

Ours: stop at *discovery*

The moment we enqueue the end, we know its parent and we can stop the whole search. The trace and tables in this lecture follow this rule.

Many books: stop at *dequeue*

The route has the same length, but the search may discover **more cells** before it ends.

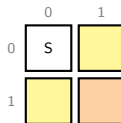
On the slip maze, with the same neighbor order:

	Dequeued	Discovered
Stop on discovery (ours)	3	5
Stop on dequeue	5	6

Neighbor order: any fixed order works. It can change which route DFS finds; BFS always finds a route with the fewest moves, when every move costs the same.

Why Mark Cells When They Are *Enqueued*?

An open 2×2 grid, $S = (0, 0)$. Cell $(1, 1)$ is a neighbor of both $(0, 1)$ and $(1, 0)$.



Mark when dequeued (wrong)

Dequeue S : enqueue $(0, 1)$, $(1, 0)$

Dequeue $(0, 1)$: $(1, 1)$ not marked yet, enqueue it

Dequeue $(1, 0)$: $(1, 1)$ *still* not marked, enqueue it **again**

Queue holds $(1, 1)$ twice

Mark when enqueued (right)

Dequeue S : mark and enqueue $(0, 1)$, $(1, 0)$

Dequeue $(0, 1)$: mark and enqueue $(1, 1)$

Dequeue $(1, 0)$: $(1, 1)$ is already marked, **skip**

Queue holds $(1, 1)$ once

Rule

Mark a cell visited the moment you put it in the queue. Then each cell enters the queue **at most once**, and its parent is set once and never overwritten.

BFS Step by Step: the Maze from Slip Question 3

	0	1	2
0	S		
1			
2	T		

Same maze as Class 11. DFS (left, right, up, down) entered 7 cells before reaching *T*.

BFS uses the same neighbor order here, so the steps are easy to compare.

Watch:

- the **queue**: front on the left;
- the **parent table**: who discovered each cell.

Orange: just dequeued. Yellow: in the queue. Blue: explored. White: not discovered. Green: final route.

Quick Question: What Type Is the Parent Table?

For every maze cell we want to remember **the cell that discovered it**. Recall the definition of a cell:

```
struct Coord {  
    int row;  
    int col;  
};
```

Which declaration gives the right type for parent?

- (A) `std::vector<std::vector<int>> parent;`
- (B) `std::vector<std::vector<Coord>> parent;`
- (C) `std::vector<std::vector<bool>> parent;`
- (D) `std::vector<Coord> parent;`

Quick Question: What Type Is the Parent Table?

For every maze cell we want to remember **the cell that discovered it**. Recall the definition of a cell:

```
struct Coord {  
    int row;  
    int col;  
};
```

Which declaration gives the right type for parent?

- (A) `std::vector<std::vector<int>> parent;`
- (B) `std::vector<std::vector<Coord>> parent;`
- (C) `std::vector<std::vector<bool>> parent;`
- (D) `std::vector<Coord> parent;`

Answer: (B). A parent is a row *and* a column, so each entry is a Coord; there is one entry per maze cell, so we need a vector of vectors. (A) holds one number, (C) holds only true/false, (D) loses the shape of the maze.

Designing the Parent Table: Five Questions

Maze

	0	1	2
0	S		
1			
2	T		

Same maze as before.

Open cells: (0,0) is *S*,
(2,0) is *T*.

- 1 **What do we store?** The cell that **discovered** this one: a row *and* a column, so a `Coord`, not an `int`.
- 2 **What shape?** One entry per maze cell, the **same shape as the maze**: `parent[row][col]`.
- 3 **What means “no parent yet”?** `Coord{-1,-1}`, not (0,0), which is a real cell.
- 4 **When do we write?** The moment a cell is **enqueued**:
`parent[nb.row][nb.col] = curr;`
- 5 **When do we read?** After we find *E*: follow the parents back to *S*.

Side by Side: Before the Search

Both tables have the **same shape as the maze**. The parent table holds a Coord, a row and a column.

	Maze			visited			parent		
	0	1	2	0	1	2	0	1	2
0	S			false	false	false	(-1,-1)	(-1,-1)	(-1,-1)
1				false	false	false	(-1,-1)	(-1,-1)	(-1,-1)
2	T			false	false	false	(-1,-1)	(-1,-1)	(-1,-1)

`visited[r][c]` is a bool. `parent[r][c]` is a Coord. Every parent entry starts as `{-1,-1}`, meaning “no parent yet”. Walls keep false and `{-1,-1}` forever.

Quick Question: Writing and Reading parent

```
struct Coord { int row; int col; };
```

1. BFS dequeued `curr = (0, 0)` and discovers `nb = (0, 1)`. Which statement records that?

- (A) `parent[nb] = curr;`
- (B) `parent[nb.row][nb.col] = curr;`
- (C) `parent[curr.row][curr.col] = nb;`
- (D) `parent[nb.row][nb.col] = curr.row;`

2. Which expression is the *row* of the cell that discovered `cell`?

- (A) `parent[cell].row`
- (B) `parent[cell.row][cell.col].row`
- (C) `parent.row[cell]`
- (D) `parent[cell.row].col`

Quick Question: Writing and Reading parent

```
struct Coord { int row; int col; };
```

1. BFS dequeued `curr = (0, 0)` and discovers `nb = (0, 1)`. Which statement records that?

- (A) `parent[nb] = curr;`
- (B) `parent[nb.row][nb.col] = curr;`
- (C) `parent[curr.row][curr.col] = nb;`
- (D) `parent[nb.row][nb.col] = curr.row;`

Answers: (B) and (B). Index with `[row][col]` of the cell being described; `.row` and `.col` come *after* both indexes.

2. Which expression is the *row* of the cell that discovered `cell`?

- (A) `parent[cell].row`
- (B) `parent[cell.row][cell.col].row`
- (C) `parent.row[cell]`
- (D) `parent[cell.row].col`

Building and Using the Two Tables in C++

```
std::vector<std::vector<bool>> visited;
std::vector<std::vector<Coord>> parent;
for (const auto& row : maze) {           // one row at a time
    visited.emplace_back(row.size(), false);
    parent.emplace_back(row.size(), Coord{-1, -1});
}

// write: when neighbor nb is discovered from curr
visited[nb.row][nb.col] = true;
parent[nb.row][nb.col] = curr;           // a whole Coord

// read: who discovered cell?
Coord p = parent[cell.row][cell.col]; // p.row and p.col
```

- Building **row by row** copies each row's length, so it works even when rows have **different lengths**. A fixed `vector<vector<Coord>> parent(rows, vector<Coord>(cols))` would not.
- `parent[r][c]` is a `Coord`; use `.row` and `.col` on it, or compare both fields.

BFS Step by Step: Step 1 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front → back)

(0,0)

Parent table

(empty)

Start: mark $S = (0,0)$ visited and enqueue it. The queue holds the cells we have discovered but not yet explored.

BFS Step by Step: Step 2 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(0,1)	(1,0)
-------	-------

Parent table

$\text{parent}(0,1) = (0,0)$

$\text{parent}(1,0) = (0,0)$

Dequeue (0,0). Neighbors: left outside; right **open**: mark, parent = (0,0), enqueue; up outside; down **open**: mark, parent = (0,0), enqueue.

Your Turn: Code for $\text{parent}(0,1) = (0,0)$

In Step 2, BFS dequeued $(0,0)$ and discovered $(0,1)$, so the parent table says $\text{parent}(0,1) = (0,0)$. Recall the definition of a cell:

```
struct Coord {  
    int row;  
    int col;  
};
```

Which code stores this in `std::vector<std::vector<Coord>>` `parent`?

- (A) `parent[0][1] = Coord{0, 0};`
- (B) `parent[0][0] = Coord{0, 1};`
- (C) `parent[0][1] = 0;`
- (D) `parent[0][1].row = 0;`

Your Turn: Code for $\text{parent}(0,1) = (0,0)$

In Step 2, BFS dequeued $(0,0)$ and discovered $(0,1)$, so the parent table says $\text{parent}(0,1) = (0,0)$. Recall the definition of a cell:

```
struct Coord {  
    int row;  
    int col;  
};
```

Which code stores this in `std::vector<std::vector<Coord>>` `parent`?

- (A) `parent[0][1] = Coord{0, 0};`
- (B) `parent[0][0] = Coord{0, 1};`
- (C) `parent[0][1] = 0;`
- (D) `parent[0][1].row = 0;`

Answer: (A). The *index* is the discovered cell $(0,1)$; the *value* is the whole cell that discovered it, $(0,0)$. In general:

`parent[nb.row][nb.col] = curr;` (B) swaps them, (C) stores an int, and (D) sets only the row.

BFS Step by Step: Step 3 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(1,0)	(0,2)
-------	-------

Parent table

parent(0,1) = (0,0)

parent(0,2) = (0,1)

parent(1,0) = (0,0)

Dequeue (0,1). Neighbors: left visited; right **open**: mark, parent = (0,1), enqueue; up outside; down wall.

BFS Step by Step: Step 4 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(0,2)	(2,0)
-------	-------

Parent table

parent(0,1) = (0,0)

parent(0,2) = (0,1)

parent(1,0) = (0,0)

parent(2,0) = (1,0)

Dequeue (1,0). Neighbors: left outside; right wall; up visited; down **open**: mark, parent = (1,0), enqueue. The end is discovered, so we can stop.

BFS Step by Step: Step 5 of 5

	0	1	2
0	S		
1			
2	T		

Queue (front $\rightarrow$ back)

(0,2)	(2,0)
-------	-------

Parent table

parent(0,1) = (0,0)

parent(0,2) = (0,1)

parent(1,0) = (0,0)

parent(2,0) = (1,0)

Rebuild the route: start at the end and follow the parents back: $(2,0) \rightarrow (1,0) \rightarrow (0,0)$. Pushing each cell on a stack as we go leaves the **start on top**.

Side by Side: After the Search

BFS stopped when it discovered $T = (2,0)$. Read the parent table at T :

	Maze			visited			parent		
	0	1	2	0	1	2	0	1	2
0	S			true	true	true	$(-1,-1)$	$(0,0)$	$(0,1)$
1				true	false	false	$(0,0)$	$(-1,-1)$	$(-1,-1)$
2	T			true	false	false	$(1,0)$	$(-1,-1)$	$(-1,-1)$

`parent[2][0]` is $(1,0)$, and `parent[1][0]` is $(0,0)$. S has $\{-1,-1\}$: nobody discovered it. So do the open cells $(1,2)$ and $(2,2)$ that BFS never reached.

Your Turn: Designing the Table

	Maze			visited			parent		
	0	1	2	0	1	2	0	1	2
0	S			true	true	true	$(-1,-1)$	$(0,0)$	$(0,1)$
1				true	false	false	$(0,0)$	$(-1,-1)$	$(-1,-1)$
2	T			true	false	false	$(1,0)$	$(-1,-1)$	$(-1,-1)$

- 1 Why is each parent entry a Coord and not an int?
- 2 Write the code that builds parent with the same shape as maze, every entry $\{-1,-1\}$. The rows of maze may have different lengths.
- 3 Why is “no parent” $\{-1,-1\}$ and not $\{0,0\}$?
- 4 BFS has dequeued $\text{curr} = (0,1)$ and finds the open, unvisited neighbor $\text{nb} = (0,2)$. Write the statements that mark nb and record its parent.

Your Turn: Reading the Table

	Maze			visited			parent		
	0	1	2	0	1	2	0	1	2
0	S			true	true	true	(-1,-1)	(0,0)	(0,1)
1				true	false	false	(0,0)	(-1,-1)	(-1,-1)
2	T			true	false	false	(1,0)	(-1,-1)	(-1,-1)

- 5 What is `parent[2][0]`? What are its `.row` and `.col`?
- 6 Start at $T = (2,0)$ and follow the parents. List the cells in the order you visit them. How does the loop know when to stop?
- 7 Besides walls, which cells hold $\{-1,-1\}$ in the table? Why does each one?
- 8 Is the parent table enough to replace `visited`? Check cell `S`.

Answers: Designing the Table

Maze			visited			parent					
	0	1	2		0	1	2		0	1	2
0	S			0	true	true	true	0	(-1,-1)	(0,0)	(0,1)
1				1	true	false	false	1	(0,0)	(-1,-1)	(-1,-1)
2	T			2	true	false	false	2	(1,0)	(-1,-1)	(-1,-1)

- 1 A parent is a cell, so it needs a row and a column; one `int` holds only one of them.
- 2 Choose the code: multiple choice on the next slide.
- 3 (0,0) is a real cell (usually *S*); -1 is never a valid row or column, so it cannot be confused with a cell.
- 4 Choose the statements: multiple choice on the slide after next.

Multiple Choice: Building parent

The maze is a `std::vector<std::vector<char>>` `maze` whose rows may have **different lengths**. We want `parent` to have the same shape, every entry `Coord{-1,-1}`. Recall:

```
struct Coord { int row; int col; };
```

Which code builds it?

- (A) `std::vector<std::vector<Coord>> parent(maze.size(), std::vector<Coord>(maze[0].size()));`
- (B) `for (const auto& row : maze)
parent.emplace_back(row.size(), Coord{-1, -1});`
- (C) `for (const auto& row : maze)
parent.emplace_back(row.size(), Coord{0, 0});`
- (D) `parent.emplace_back(maze.size(), Coord{-1, -1});`

Multiple Choice: Building parent

The maze is a `std::vector<std::vector<char>>` maze whose rows may have **different lengths**. We want parent to have the same shape, every entry `Coord{-1,-1}`. Recall:

```
struct Coord { int row; int col; };
```

Which code builds it?

- (A) `std::vector<std::vector<Coord>> parent(maze.size(), std::vector<Coord>(maze[0].size()));`
- (B) `for (const auto& row : maze)
 parent.emplace_back(row.size(), Coord{-1, -1});`
- (C) `for (const auto& row : maze)
 parent.emplace_back(row.size(), Coord{0, 0});`
- (D) `parent.emplace_back(maze.size(), Coord{-1, -1});`

Answer: (B). It copies each row's own length and uses the sentinel `{-1,-1}`. (A) uses only row 0's length; (C) uses a real cell; (D) builds a single row.

Multiple Choice: Recording a Discovery

BFS has dequeued $\text{curr} = (0, 1)$ and finds the open, unvisited neighbor $\text{nb} = (0, 2)$.

Which statements are correct?

- (A) `visited[curr.row][curr.col] = true;`
`parent[curr.row][curr.col] = nb;`
- (B) `visited[nb.row][nb.col] = true;`
`parent[nb.row][nb.col] = curr; que.push(nb);`
- (C) `visited[nb.row][nb.col] = true;`
`parent[curr.row][curr.col] = nb;`
- (D) `parent[nb.row][nb.col] = curr;`

Multiple Choice: Recording a Discovery

BFS has dequeued $\text{curr} = (0, 1)$ and finds the open, unvisited neighbor $\text{nb} = (0, 2)$.

Which statements are correct?

- (A) `visited[curr.row][curr.col] = true;`
`parent[curr.row][curr.col] = nb;`
- (B) `visited[nb.row][nb.col] = true;`
`parent[nb.row][nb.col] = curr; que.push(nb);`
- (C) `visited[nb.row][nb.col] = true;`
`parent[curr.row][curr.col] = nb;`
- (D) `parent[nb.row][nb.col] = curr;`

Answer: (B). Mark nb (the cell being discovered), record who discovered it (curr), and enqueue it. (A) and (C) record the parent on the wrong cell; (D) never marks nb visited or enqueues it.

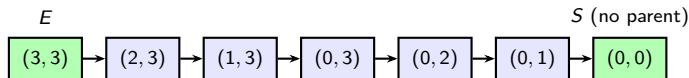
Answers: Reading the Table

	Maze			visited			parent		
	0	1	2	0	1	2	0	1	2
0	S			true	true	true	$(-1,-1)$	$(0,0)$	$(0,1)$
1				true	false	false	$(0,0)$	$(-1,-1)$	$(-1,-1)$
2	T			true	false	false	$(1,0)$	$(-1,-1)$	$(-1,-1)$

- 5 $(1,0)$: `.row` is 1 and `.col` is 0.
- 6 $(2,0) \rightarrow (1,0) \rightarrow (0,0)$. Stop when the parent is $\{-1,-1\}$: that is S .
- 7 S (nobody discovered it) and the open cells $(1,2)$ and $(2,2)$ (BFS stopped before reaching them).
- 8 No: S is visited, but its parent is $\{-1,-1\}$, the same as an undiscovered cell.

Rebuilding the Route: Parents Form a Linked List

On the 4×4 maze, BFS records one parent per discovered cell. Start at E and follow the parents:



- A **singly linked list**: each cell's "next" is its parent; the list ends at S .
- We reach the cells from end to start, but the answer needs the **start on top**. A **stack** reverses the order (like printing a string backward): push E first, S last.
- If E was never discovered, return an **empty stack**.

	0	1	2	3
0	S (0,0)	(0,1)	(0,2)	(0,3)
1				(1,3)
2				(2,3)
3				E (3,3)

The route in the maze, with coordinates

DFS vs. BFS: Side by Side

	DFS (recursive)	BFS
Container	Call stack	<code>std::queue</code> (explicit)
Search order	Deep along one branch, then back up	Layer by layer
Route found	First one found; 9 cells here	A route with the fewest cells; 7 cells here
Backtracking	Yes: a return	None: nothing is ever undone
Route is built by	Unwinding the calls (answer stack)	Following parents from <i>E</i> to <i>S</i>
Neighbor order	Matters: it decides the route	Any order: same length
4×4 example	Enters 11 cells	Marks 9 cells (including <i>S</i> and <i>E</i>)

Time and Space

Let n be the total number of cells in the maze.

	DFS	BFS
Time	$O(n)$: each cell is entered once, with at most 4 neighbor checks	$O(n)$: each cell is enqueued and dequeued at most once, with at most 4 neighbor checks
Extra space	Visited table $O(n)$; call stack up to $O(n)$ deep in a long winding maze	Visited and parent tables $O(n)$; queue holds the current frontier, at most $O(n)$

Take-away

Both visit each cell at most once, so both are linear in the worst case. The main difference is **which route you get**; the actual work also depends on the search order and on where the search stops.

Which One Should I Use?

Use BFS when...

- you need the route with the **fewest steps** (unweighted);
- “what is the distance from S to E ?”

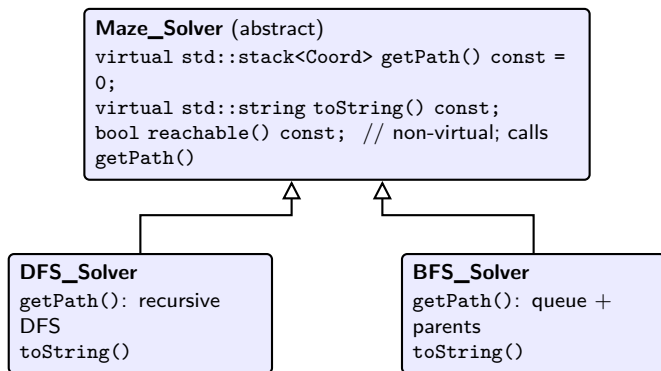
Use DFS when...

- **any** route is enough, or you want to explore *every* possibility;
- the problem is naturally recursive and you try choices and **undo** them.

Looking ahead

After the midterm, Project 3 (the Make 24 puzzle) tries choices, recurses, and backs up when a choice fails: recursion and **backtracking**, the DFS pattern in a new setting.

One Interface, Two Solvers



`reachable()` is written **once** in the base class and is **not** virtual: it runs the base version, `!getPath().empty()`. That call to `getPath()` *is* virtual, so it runs the DFS or BFS version.

Your Turn: What Prints If virtual Is Missing?

```
struct Maze_Solver {  
    std::string name() const { return "base"; } // NOT virtual  
};  
struct BFS_Solver : Maze_Solver {  
    std::string name() const { return "BFS"; }  
};  
BFS_Solver bfs;  
Maze_Solver* p = &bfs;  
std::cout << p->name();
```

What does the last line print?

- (A) BFS
- (B) base
- (C) BFSbase
- (D) It does not compile.

Your Turn: What Prints If virtual Is Missing?

```
struct Maze_Solver {  
    std::string name() const { return "base"; } // NOT virtual  
};  
struct BFS_Solver : Maze_Solver {  
    std::string name() const { return "BFS"; }  
};  
BFS_Solver bfs;  
Maze_Solver* p = &bfs;  
std::cout << p->name();
```

What does the last line print?

- (A) BFS
- (B) **base**
- (C) BFSbase
- (D) It does not compile.

Answer: (B). Without `virtual`, the call is decided by the **pointer type** (`Maze_Solver*`) at compile time. With `virtual` `std::string name() const`, it would print BFS.

Calling Through a Base-Class Pointer

```
DFS_Solver dfs(room, start, end);
BFS_Solver bfs(room, start, end);
Maze_Solver* solvers[] {&dfs, &bfs};

for (const Maze_Solver* curr : solvers) {
    std::cout << curr->toString() << '\n';
    // getPath() (virtual) runs the version of the REAL
    // object; reachable() is the base version, which
    // calls getPath() virtually
}
```

- The pointer type is `Maze_Solver*`, but the object is a `DFS_Solver` or a `BFS_Solver`. A **virtual** call is decided at **run time** by the real object.
- The loop does not know, or care, which search it is running. This is Task D of Project 2.
- Without `virtual`, the call would always run the base-class version.
- The base class declares a **virtual destructor** so that deleting through a base pointer is safe.

Project 2, Task B: Checklist

- Use `std::queue<Coord>`. Mark cells visited when **enqueued**.
- Keep a **parent** table with the same shape as the maze (rows may have **different lengths**).
- Stop when the end is discovered, as in today's trace (stopping at dequeue gives the same route length).
- Rebuild the route by following parents from the end to the start, pushing each cell on a `std::stack<Coord>` so the **start is on top**.
- Edge cases: start or end is a wall or outside the maze; empty maze; start equals end (one-cell route); no route $\Rightarrow$ **empty stack**.
- Check with `valgrind`: no leaks.

Summary

- **DFS** goes deep and returns the first route it finds. **BFS** spreads out in layers and finds a route with the **fewest cells**.
- BFS uses a **queue**; mark a cell when it is **enqueued**; record a **parent** for each cell.
- The route is rebuilt by following **parents** (a linked list) from the end to the start; a **stack** puts the start on top.
- Both searches are $O(n)$ time in the worst case; the main difference is the **route**, though search order and early stopping change how much work is actually done.
- `DFS_Solver` and `BFS_Solver` share one interface through a virtual `getPath()`.

Quick Check

Maze

	0	1	2
0	S		
1			
2	T		

The maze from the slip: $S = (0, 0)$,
 $T = (2, 0)$

- 1 What is in the queue (a) right after $(0, 1)$ is dequeued, (b) after its neighbors are processed?
- 2 This maze has no cycle, so use the 2×2 grid on slide 10: why would marking only when dequeued enqueue $(1, 1)$ twice?
- 3 BFS found T through the parents $(2, 0) \rightarrow (1, 0) \rightarrow (0, 0)$. Why does the answer stack have the *start* on top?
- 4 What are `parent[2][0]` and `parent[0][0]`? What does the second one mean?
- 5 Which of DFS or BFS can return a route that is not the shortest, and why?
- 6 What does `virtual` change in `curr->toString()`?